AI-Native Compliance Infrastructure

Project overview, current status, and what an entrepreneur would build next

October 2026 - working prototype - built on public data only

Disclaimer: independent personal project for self-learning. Not affiliated with any employer, state agency or vendor. Uses only public documents and synthetic data. No internal or confidential data and no personal data (no SSNs, member or patient records, PHI/PII). Views are my own.

1. What it is

A tool that compares versions of Medicaid policy documents, detects what changed, works out which internal procedures (SOPs) are affected, and drafts the updated procedure text with a citation to the policy. Nothing changes without a named reviewer's approval, and every action is logged.

It started as a language-model pipeline on a free Kaggle GPU (Qwen2.5-7B, sentence-transformers, FAISS). Testing showed the model was unreliable at exact work, so the checkable decisions now come from a rule-based engine, with a language model used only for narrow yes/no questions about meaning. It runs as a command-line agent and as an MCP server. Only public CMS documents and synthetic SOPs are used.

2. What is done

Part	What it does	Status
Language-model pipeline	Ingest, retrieve, detect changes, map impact, draft, flag invented sentences	**Done**
Evaluation harness	18 hand-written cases with answer keys written before any run: 9 synthetic, 1 real public pair (MITA 2.0 to 3.0), 8 blind	**Done**
Rule-based engine	Decides changes, affected SOPs and drafts by rules; same answer every run	**Done**
Compliance agent	Scans folders of policies and SOPs, writes suggested changes as diffs, approval-gated edits with backup and audit log	**Done**
MCP server	Seven tools, usable from any MCP-capable assistant	**Done**
Scale tests	Generators for thousands of labeled cases and for whole policy manuals	**Done**
Language-model check	Narrow yes/no questions where rules cannot judge meaning; never writes text	**Partly** - built and safety-tested, accuracy not yet measured

Results

Version	Test	Result
Language model	10 development cases, 3 repeated runs	6 of 10 passed: found every change, but false alarms, removed requirements kept, answers varied run to run
Rule engine	10 development cases	10 of 10, identical across runs
Rule engine	8 blind cases, written after the engine	8 of 8 (one gap found on the first run, then fixed)
Rule engine	4,000 generated cases, four seeds	97.6% to 98.3%

Version	Test	Result
Full agent	200 generated policy manuals, real folder scan	96.7% to 98.5% of checks

3. Problems found and fixed

Every fix below came from a failed test or a real run. Full details are in docs/LESSONS_LEARNED.md.

#	Problem	Fix
1	Same input gave different answers from run to run.	Ran every case three times to measure it, then moved exact work (numbers, dates, edits) from the language model to rules that give the same answer every run.
2	Prompts tuned on a failing case fixed that case but not its twin.	Added holdout and blind cases the rules were never tuned on, with answer keys written before any run.
3	An unaffected SOP was flagged because the model wrote that it was not affected.	Structured JSON output and a parser that understands negation.
4	The model invented sentences the policy never asked for.	A flagger shows them to the reviewer; the rule engine now copies policy text word for word and never invents.
5	False alarms on wording-only changes.	The engine ignores rewording when the numbers and meaning words match; the control case now passes.
6	Drafts kept requirements the policy had removed.	The engine cuts the dropped clause from the SOP and flags the edit.
7	The real MITA change (eight to ten business areas) was hidden in a figure caption.	Counted things are compared across the whole document, not sentence by sentence.
8	Long documents ran out of GPU memory.	Section-by-section comparison and capped summaries; short documents are compared whole.
9	Messy PDF text (page headers, page numbers, bullets glued together) created fake changes.	Text cleaning, heading handling and better sentence splitting; whole-manual scores rose from 74% to 96.7%-98.5%.
10	SOPs that shared one common word were wrongly flagged.	Matches are ranked; only SOPs with at least 60% of the best match's overlap are kept.
11	A month change (January to October) was missed in the blind test.	Month names are now treated as values, like numbers and dates.
12	A test passed a broken draft ('mail a within 10 days').	Tests now check the exact text of every draft, not just that certain words are gone.
13	A rescan overwrote the earlier report.	Reports accumulate across scans; only new version pairs are compared.
14	Matching timed out on a library of 1,600 SOPs.	Word sets are indexed once per run.
15	After a requirement was removed, lines that depended on it stayed in (for example 'keep the mailing receipt' after the paper copy was dropped).	Those lines are now flagged for the reviewer to delete, never cut automatically.
16	About 15% of removal drafts read badly ('Obtain a and dispense the refill').	The cut now takes the whole phrase (verb, article and trailing 'to the agency' part); if it still cannot be cut cleanly, nothing changes and the SOP is flagged for a person. Broken drafts: 144 of 976 before, 0 after.

4. Honest assessment

Question	Answer today
Does it work end to end?	Yes. The agent scans folders, drafts changes, waits for approval and logs every action; covered by end-to-end tests.
Is it accurate?	On its own tests, yes. But the rules were tuned on the development cases, the generated tests use my own templates, and the blind set is now partly used. Fresh, independent cases are needed.
Does it work on real documents?	One real public pair (MITA 2.0 to 3.0) passes. No second real pair and no real SOPs yet.
Will someone pay for it?	Unknown. A first practitioner said the current manual process works and nothing is missed.
Is it defensible?	Not yet. The approval gate and audit log are the start of an audit trail, but the product is still close to 'rewrite the SOP'.

The core strategic problem: if the product is only 'rewrite the SOP', the objection 'that is small work' is right. The value has to be what surrounds the rewrite - knowing a change happened, proving it was handled, and keeping an audit trail. That is where the additions below point.

5. What I would add, in priority order

#	Addition	Why it matters	Status
1	Evaluation harness: hand-labelled policy changes, precision and recall, rerun on every change	Without numbers you have a demo. With numbers you have evidence, and you see where it fails.	**Done**: 18 hand-written cases plus generated tests
2	Automatic monitoring of public sources (CMS, Federal Register, State Medicaid Director letters, state bulletins) with alerts	Turns a tool you run into a service that runs itself. Needs no customer data.	**Not started** (2-3 weeks)
3	Obligation extraction and deadlines: each 'must', effective date and responsible party; severity	'You must do X by date Y' is what a compliance lead acts on.	**Partly**: added, removed and changed requirements are detected; no deadlines, owners or severity
4	Traceability matrix: policy clause to SOP to owner to status, with audit-ready export	This is the audit evidence and the strongest reason to pay.	**Partly**: change-to-SOP report and audit log; no owner, status or export
5	Review workflow: approve or reject each draft, tracked-changes Word export, comments, tickets	Matches how compliance teams already work.	**Partly**: named approval, backup and log; no Word export, comments or tickets
6	Citation verification: no draft text that is not in the source	Turns 'we flag invented text' into 'invented text cannot get through'.	**Done** by design: the engine only swaps values, cuts dropped clauses, or copies policy text word for word
7	Multi-state comparison: the same rule across states, side by side	Public data only; sellable to vendors and MCOs in several states.	**Not started** (3-4 weeks)
8	Security and deployment: private deployment, no data retention, audit logs, role-based access; SOC 2 and HIPAA roadmap	Medicaid buyers will not touch a tool without this.	**Partly**: runs locally with an audit log; no access control or hosted deployment

6. Possible wedges - pick one, do not build all

Wedge	Buyer	Pro	Con
A. Change-intelligence feed (monitoring + summaries + impact tags)	Vendors, MCOs, consultancies	Public data only, fast to launch, easy to test willingness to pay	Competes with existing regulatory-news products
B. SOP impact mapping and drafting (current build)	Compliance teams at plans and vendors	Matches what is built and tested	Needs customer SOPs, so slow trust and sales cycle; the 'small work' objection
C. Contract and RFP requirement tracking: map contract deliverables and modifications to what was delivered	Vendors serving state agencies	Contract changes are expensive and poorly tracked	Needs customer documents; narrower market
D. Audit-readiness pack: prove for any rule when it changed, who reviewed, what was updated	Compliance and audit leads	Strongest pain point; recurring need	Needs A and B first

My recommendation: start with A as the front door (no customer data needed, quick to demo, tests whether anyone cares), use B as the second step for design partners, and grow toward D, which is the defensible product.

7. Business side - what needs answering before more code

- Customer interviews first: 5-8 people across agencies, vendors, MCOs and other states. Ask how often a change is missed late, what that cost, and what they use today. One conversation so far.
- Competition. Regulatory-change products already exist in banking and general compliance. Find out who does Medicaid-specific work, what they miss, and why a buyer would switch.
- Pricing hypothesis. Test per-state or per-program annual pricing in interviews. Do not guess a number before someone says what a missed change costs them.
- Design partners. Aim for 2-3 organisations willing to run a free pilot in exchange for feedback and a reference.

8. 30 / 60 / 90 day plan

Window	Goal	Done when	Progress
Days 1-30	Validate	8 interviews; 15 labelled changes; accuracy numbers; decision on wedge	18 labelled cases and accuracy numbers done; 1 of 8 interviews; wedge not decided
Days 31-60	Make it real	Runs on real public versioned documents; monitoring and obligation extraction; citation verification	One real pair passes; citation problem solved by design; monitoring not started
Days 61-90	Prove it	One design-partner pilot or 3 letters of interest; traceability matrix demo	Not started

9. Risks

- No real demand. Mitigation: interviews before more engineering.
- Errors in a high-stakes domain. Mitigation: rules for checkable decisions, approval before any change, audit log, published test results including failures.
- Trust and data access. Mitigation: private deployment, public-data wedge first.

- Incumbents. Mitigation: Medicaid specialisation, audit trail as the moat.
- Limited time. Mitigation: tight scope, one wedge.

Repo: github.com/shubhamkumarsingh91-lgtm/Compliance-AI. No proprietary, client or employer data is used in this project.